# Tecnologias de Desenvolvimento de Interface Gráfica

Aula 3: JavaScript

# Agenda

- Variáveis, Funções, Objetos
- Strings, Números, Datas e Arrays
- ES6, Hoisting, Arrow Functions, Strict Mode
- Rest, Spread
- Callbacks x Promises
- Fetch Data

# Variáveis

# Variáveis

- Armazenamento de informações
- Uso da palavra reservada `var`
- Padrão "camel case"
- São case-sensitive

# Exemplos

```
var modeloDoCarro = "Fusca";

var soma = 7 + 8 + 9;

var media = soma/3;

var pessoa; // undefined

pessoa = "Joe";
```

# Exemplos

```
var pessoa = "Mary", marcaDoCarro =
"Volvo", preco = 200;
```

```
var pessoa = "Mary",

marcaDoCarro = "Volvo",

preco = 200;
```

# Funções

# Funções

- Blocos de código com escopo local
- Recebem ou não parâmetros
- Retornam ou não resultados

# Exemplo

```
function toCelsius(fahrenheit) {
  return (5/9) * (fahrenheit-32);
}
document.getElementById("demo").innerHTML =
toCelsius;
```

# Exemplo

```
// marcaDoCarro não está acessível

function minhaFuncao() {

  var marcaDoCarro = "Volvo";

  // // marcaDoCarro está acessível

}

// marcaDoCarro não está acessível
```

# Funções como Variáveis

- Funções podem ser armazenadas em variáveis

```
var x = function (a, b) {return a * b};

var z = x(4, 3);
```

# Funções Específicas

- Alerta para o usuário

```
alert(message)
```

- Depuração

```
console.log(message)

console.info(message)

console.error(message)
```

# Funções Específicas

- Manipulação de conteúdo

```
document.getElementById(identifier)
```

- Impressão

```
window.print()
```

- Localização

```
window.location()
```

# Objetos

# Objetos

- Funcionam como um dicionário de valores
- Conjunto de pares chave:valor

# Exemplo

```
var pessoa = {
    primeiroNome: "Joãozinho",
    ultimoNome: "Silva",
    idade: 50,
    corOlhos: "azul"
};
```

# Exemplo

```
var pessoa = new Object();

pessoa.primeiroNome = "Joãozinho",

pessoa.ultimoNome = "Silva";

pessoa.idade = 50;

pessoa.corOlhos = "azul";
```

# Strings, Números, Datas e Arrays

# Exemplos

- W3Schools

# Variáveis, ES6 e Hoisting

# Variáveis no ES6

- Uso da palavras reservadas `let` e `const`
- Referência aos escopos local e global
  - `let` para escopos locais
  - `var` para escopos globais
- `const:` Referência para valores constantes

# Exemplos

```
var x = 10;
// Here x is 10
{
  let x = 2;
  // Here x is 2
}
// Here x is 10
```

# Exemplos

```
const PI = 3.1415926535897 93;

PI = 3.14;        // error

PI = PI + 10;     // error

// incorrect

const PI;

PI = 3.14159265359;
```

# Exemplos

```
const pessoa = {
    primeiroNome: "Joãozinho",
    ultimoNome: "Silva"
};
pessoa.ultimoNome = "Sousa";
```

# Hoisting

- Basicamente, é a uso de variável antes da declaração
- Hoisting = Içamento
- `let` e `const` não podem ser içadas

# Exemplos

```
x = 5; // Assign 5 to x

// Find an element

elem = document.getElementById("demo");

elem.innerHTML = x;

// Display x in the element

var x; // Declare x
```

# Arrow Functions

# Arrow Functions

- Modo enxuto de escrita de funções anônimas
- Introduzido a partir do ECMAScript 6 (ES6)
- Arrow (=>) ao invés da palavra reservada function
- Torna o código menos verboso
- Ajuda a detecção de escopo e contexto

# Exemplos

```
// ES5
var x = function(x, y) {
  return x * y;
}
// ES6
const x = (x, y) => x * y;
```

# Strict Mode

# Strict Mode

- Variante restrita que pode deixar o código mais rápido
  - Lança exceções para erros “silenciosos” do JavaScript
- Ver: W3Schools e Mozilla

# Rest

# Rest

- Flexibilização do número de parâmetros em uma função
- Uso de `arguments` (Object)
- Operador `...args` (args sendo um Array)

# Exemplos

```
function sum(n1, n2) {
    console.log(arguments);
}

function sum(...args) {
    console.log(args);
}

sum(1,2,3,4,5,6);
```

# Exemplos

```
const sum2 = (n1, n2, ...args) => {
    console.log(n1, n2, ...args);
}
sum2(1,2,3,4,5,6);
```

# Exemplos

```
function sum3(...args) {

    return args.reduce((acc, value) =>
acc+value,0);

}

console.log(sum3(1,2,3,4,5,6));
```

# Spread

# Spread

- Flexibilização de passagem de parâmetros para uma função
- Mesmo operador Rest, lógica inversa
  - Transforma um iterável em parâmetros distintos
  - String, Array, Literals, Iteráveis

# Spread

- Gerar ou combinar novos arrays
- Código menos verboso

# Exemplos

```
// sem spread

const multiply = (...args) => args.reduce((acc, v) =>

        acc * v, 1);

const sum4 = (...rest) => multiply.apply(undefined,
rest);

console.log(sum4(1,2,3,4,5,6));
```

# Exemplos

```
// com spread

const multiply = (...args) => args.reduce((acc, v) =>

        acc * v, 1);

const sum5 = (...rest) => multiply(...rest);

console.log(sum5(1,2,3,4,5,6));
```

# Callbacks X Promises

# Callbacks X Promises

- Funções de callback são utilizadas após a execução de uma tarefa síncrona

- Considere os exemplos a seguir:

# Exemplos

```
function doSomething(callback) {
    setTimeout(function() {
        callback('First Data');
    }, 1000);
}
```

```
function doOtherSomething(callback) {
    setTimeout(function() {
        callback('Second Data');
    }, 1000);
}
```

# Exemplos

```
function doAll() {
    doSomething(function(data) {
        var procData = data.split('');
        doOtherSomething(function(data2) {
            var procData2 = data2.split('');
            setTimeout(function() {
                console.log(procData, procData2);
            }, 1000);
        });
    });
}
doAll();
```

# Callbacks X Promises

- Imagina a seguinte situação:
  - Se nessas funções, algum desses dados não estiverem presentes
  - Os dados precisarem de tratamento em cada etapa
- Veja como poderia ficar o código...

```
function doAllHell() {
    try {
        doSomething(function(data) {
            var procData = data.split('');
            try {
                doOtherSomething(function(data2) {
                    var procData2 = data2.split('');
                    try {
                        setTimeout(function() {
                            console.log(procData, procData2);
                        }, 1000);
                    } catch (err) {
                        //handle error
                    }
                });
            } catch (err) {
                // handle error
            }
        });
    } catch (err) {
        // handle error
    }
}
```

# Callbacks X Promises

- Esse é o cenário do “callback hell”
- Na solução desse problema, escrevemos Promises
  - Pending
  - Fulfilled
  - Rejected

```
const doSomethingPromise = new Promise((resolve, reject) => {
    // throw new Error('Something went wrong');
    setTimeout(function() {
        resolve('First Data');
    }, 1000);
});

const doOtherSomethingPromise = new Promise((resolve, reject) => {
    setTimeout(function() {
        resolve('Second Data');
    }, 1000);
});

// chamada com then e catch encadeados
doSomethingPromise
    .then(data => console.log(data.split('')))
    .catch(error => console.log(error));

doSomethingPromise
    .then(data => doOtherSomethingPromise)
    .then(data2 => console.log(data2.split('')))
    .catch(error => console.log(error));
```

```
const doSomethingPromise = () => new Promise((resolve, reject) => {
    // throw new Error('Something went wrong);
    setTimeout(function() {
        resolve('First Data');
    }, 1000);
});

const doOtherSomethingPromise = () =>
    new Promise((resolve, reject) => {
        setTimeout(function() {
            resolve('Second Data');
        }, 1000);
    });

// processamento sequencial
doSomethingPromise()
    .then(data => {
        console.log(data.split(''));
        return doOtherSomethingPromise();
    })
    .then(data2 => console.log(data2.split('')))
    .catch(error => console.log(error));
```

```
const doSomethingPromise = () => new Promise((resolve, reject) => {
    // throw new Error('Something went wrong);
    setTimeout(function() {
        resolve('First Data');
    }, 1000);
});

const doOtherSomethingPromise = () =>
    new Promise((resolve, reject) => {
        setTimeout(function() {
            resolve('Second Data');
        }, 1000);
    });

// proc. paralelo (all - sujeita a erro; race - resolve primeiro)
Promise.all([doSomethingPromise(), doOtherSomethingPromise()])
        .then(data => console.log(data));

Promise.all([doSomethingPromise(), doOtherSomethingPromise()])
        .then(data => {
            console.log(data[0].split(''));
            console.log(data[1].split(''));
        });
```

# Fetch Data

# Fetch Data

- Consumo de dados externos (JSON, XML, Files)
- Uso da biblioteca XMLHTTPRequest (callbacks)
- API fetch evita o “callback hell”
- Tratativas com Streams

# Exemplos

```
fetch('https://demo0926641.mockable.io/data')
    .then(responseStream => {
        responseStream.json().then(data => {
            console.log(data);
        });
    });
```

# Exemplos

```
fetch('https://demo0926641.mockable.io/data')
    .then(responseStream =>
responseStream.json())
    .then(data => {
        console.log(data);
    }).catch(error => {
        console.log(error);
    });
```

# Exemplos

```
fetch('https://demo0926641.mockable.io/data', {
    method: 'post',
    body: JSON.stringify({"data":"example"})
})
.then(responseStream => responseStream.json())
.then(data => {
    console.log(data);
}).catch(error => console.log(error));
```

# Fetch Data

- ES7 introduz o conceito de async/await
- Exemplo de função resolvida

```
const myFunc = async () => 12345;

console.log(myFunc());

myFunc().then(data => console.log(data));
```

# Exemplos

```
const myFunc2 = async () => {
    const data = await doSomethingPromise();
    return data;
}

console.log(myFunc2());

myFunc2().then(data => console.log(data));
```

# Obrigado!